

고려아연 WordPress 보안 점검 및 개선 최종 보고서

대상: C:\ko-release 내 프로그램 구동 관련 PHP, HTML, JS 파일. PDF 파일은 분석 대상에서 제외.

최종 결론

명령 실행형 악성코드나 웹셸로 단정할 코드는 확인되지 않았습니다. 그러나 커스텀 플러그인과 테마에 비로그인 AJAX 공개, 관리자성 AJAX 권한 검증 누락, 파일 업로드/다운로드 검증 부족이 존재합니다. 특히 `admin-ajax.php?action=customer_list` 는 비로그인 상태에서 민원 목록 데이터가 반환되는 것이 실제로 확인되었습니다.

1. 위험 등급 기준 재정의

기존 초안의 "긴급" 표기는 일부 항목에서 과도하게 보일 수 있어 다음 기준으로 재분류했습니다. 단순 정보 노출은 원칙적으로 **높음**으로 분류하고, 서버 파일 접근·악성 파일 배포·권한 상승처럼 시스템 장악 또는 운영 변경으로 이어질 수 있는 항목만 **긴급**으로 분류합니다.

등급	판단 기준	이번 점검 적용 예시
긴급	서버 파일 탈취, 악성 파일 배포, 관리자 기능 변조, 권한 상승 등 운영 시스템에 직접 영향 가능	<code>download_file</code> 기반 임의 파일 다운로드 가능성, 검증 부족 업로드로 인한 악성 파일 저장/배포 가능성, 일반 로그인 사용자에게 의한 관리자성 AJAX 호출
높음	비로그인 민감정보 노출, 개인정보·민원 상세·첨부 경로 노출, 중요 업무 데이터 조회	<code>customer_list</code> 비로그인 민원 목록 노출
중간	CSRF, 입력값 검증 미흡, 공개 기능 남용 가능성, 제한적 정보 노출	공개 문의 등록 API의 rate limit/CAPTCHA 부재, 공개 추가 API 호출 남용 가능성
낮음	운영 정보 노출, 방어 심화 필요, 단독 악용 가능성 낮음	불필요한 테스트 파일, 로그 정책 미흡

2. 주요 발견사항 요약

등급	대상	문제점	권장 조치
높음	<code>btr_customer_inquiry.php</code> <code>customer_list</code>	<code>wp_ajax_nopriv_customer_list</code> 로 등록되어 비로그인 호출이 가능하며, 권한 검증 코드가 주석 처리되어 있습니다. 실제 호출 결과 민원 목록과 연락처, 상세 내용, 첨부 경로 등이 반환되었습니다.	비로그인 혹 제거. 관리자 또는 지정 담당자만 조회 가능하도록 <code>current_user_can()</code> 또는 서버 검증 토큰을 DB 조회 전 강제.
긴급	<code>btr_admin_functions.php</code> <code>download_file</code>	사용자가 전달한 URL/경로를 서버 경로로 변환한 뒤 <code>readfile()</code> 로 내려주는 구조입니다. 경로 검증이 약해 임의 파일 다운로드로 이어질 수 있습니다.	핸들러 제거 또는 업로드 디렉터리 내부의 허용 파일만 다운로드하도록 재작성. 가능하면 DB에 등록된 첨부 ID 기반 다운로드로 단일화.

등급	대상	문제점	권장 조치
긴급	문의/신고 첨부 업로드	공개 AJAX에서 파일 업로드와 base64 파일 저장이 가능하며, 확장자·MIME·용량·실행 가능 확장자 차단이 충분하지 않습니다.	서버 측 allowlist, MIME 검증, 용량 제한, 무작위 파일명, 실행 확장자 차단, 업로드 폴더 PHP 실행 금지 적용.
긴급	<code>btr_utility</code> , <code>btr_social</code> 관리자성 AJAX	<code>wp_ajax_*</code> 는 로그인만 요구합니다. 콜백 내부에 권한 검증과 nonce 검증이 없어 일반 로그인 사용자가 팝업, 배당정보, 태그, SNS 데이터, 토큰 발급 기능을 직접 호출할 수 있습니다.	모든 관리자성 AJAX 시작부에 <code>check_ajax_referer()</code> 와 <code>current_user_can('edit_others_posts')</code> 또는 <code>manage_options</code> 적용.
높음	<code>customer_detail</code> , <code>admin_result</code>	관리자성 또는 민원 상세 기능이 <code>wp_ajax_nopriv_*</code> 로도 등록되어 있습니다. 일부 내부 권한 검증이 있으나 외부 공개 라우트로 열어 둘 필요가 낮습니다.	<code>nopriv</code> 혹 제거. 외부 시스템 연동이 필요하면 전용 API 키 또는 OAuth 토큰을 DB 처리 전 검증.
중간	공개 문의 등록/검색/삭제	비로그인 등록, 비밀번호 기반 조회/삭제 구조가 존재합니다. CAPTCHA, rate limit, 잠금 정책이 없으면 스팸·무차별 대입·대량 등록에 취약합니다.	CAPTCHA, IP/세션 기반 rate limit, 실패 횟수 제한, 비밀번호 해시 저장, 응답 필드 최소화 적용.
중간	관리자 화면 JS 출력	민원 제목/상세/첨부 URL 등 사용자 입력값을 HTML 문자열로 조합해 출력하는 부분이 있어 저장형 XSS 가능성이 있습니다.	서버 출력 시 <code>esc_html()</code> , <code>esc_url()</code> 적용. JS에서는 <code>.html()</code> 대신 <code>.text()</code> 또는 DOM API 사용.
낮음	<code>phpinfo_temp.php</code>	<code>phpinfo()</code> 테스트 파일이 남아 있어 서버 환경 정보가 노출됩니다.	즉시 삭제.

3. admin-ajax.php 및 admin-post.php 검토 결과

`wp-admin/admin-ajax.php` 와 `wp-admin/admin-post.php` 는 WordPress 코어 라우터입니다. 라우터 자체가 모든 업무 권한을 판단하지 않고, 등록된 action의 콜백 함수가 권한 검증을 수행해야 합니다.

파일	역할	이번 점검 판단
<code>wp-admin/admin-ajax.php</code>	<code>wp_ajax_*</code> , <code>wp_ajax_nopriv_*</code> 요청을 각 콜백으로 전달	실제 악용 URL의 입구입니다. 예: <code>/wp-admin/admin-ajax.php?action=customer_list</code> . 다만 수정 대상은 코어 파일이 아니라 커스텀 플러그인의 action 등록과 콜백 권한 검증입니다.
<code>wp-admin/admin-post.php</code>	<code>admin_post_*</code> , <code>admin_post_nopriv_*</code> 요청 처리	커스텀 코드에서 연결된 <code>admin_post_*</code> 혹은 발견되지 않았습니다. 현재 확인된 취약점의 직접 경로는 아니지만, 향후 혹 추가 시 동일한 권한 검증 원칙을 적용해야 합니다.

4. 권한 검증 누락 상세 및 수정 제안

4.1 비로그인 민원 목록 노출: customer_list

파일: wp-content/plugins/btr_customer_inquiry/btr_customer_inquiry.php

문제: customer_list() 내부의 권한 검증이 주석 처리되어 있고, wp_ajax_nopriv_customer_list 가 활성화되어 있습니다. 이로 인해 비로그인 사용자가 division 파라미터만 알면 민원 목록을 조회할 수 있습니다.

기존 코드 - 비로그인 AJAX 공개

```
// add_action('wp_ajax_customer_list', 'customer_list');
add_action('wp_ajax_nopriv_customer_list', 'customer_list');
```

권장 수정:

변경 권장 코드 - 비로그인 혹 제거 및 권한 검증 추가

```
// 관리자 또는 지정 담당자 로그인 사용자만 허용
add_action('wp_ajax_customer_list', 'customer_list');
// add_action('wp_ajax_nopriv_customer_list', 'customer_list');

function customer_list() {
    // DB 조회 전에 권한을 먼저 검증
    if (!current_user_can('administrator') && !btr_current_user_can_view_inquiry()) {
        wp_send_json_error(['message' => '접근 권한이 없습니다.'], 403);
    }

    // 관리자 화면 AJAX라면 nonce도 함께 검증
    check_ajax_referer('btr_admin_nonce', 'nonce');

    // 권한 확인 후 division 검증 및 DB 조회 수행
}
```

4.2 일반 사용자 권한으로 관리자 기능 호출 가능

대상 파일: btr_utility/btr_main_setting.php, btr_social/btr_social.php

문제: 아래 기능은 wp_ajax_* 로 등록되어 있어 로그인 사용자는 라우터를 통과할 수 있습니다. 그러나 콜백 내부 권한 검증이 없어 일반 사용자 계정으로 관리자 기능을 호출할 가능성이 있습니다.

- btr_popup_save, btr_popup_delete, btr_popup_edit : 메인 팝업 변경
- btr_dividends_profit_save, btr_dividends_profit_delete, btr_dividends_profit_edit : 배당/실적 정보 변경
- btr_token_get_and_store_token : 외부 API 토큰 발급 및 저장
- btr_tag_save, btr_tag_edit, btr_tag_delete, update_tag_order : 태그 변경
- social_data_save, edit_social_data, social_media_data_delete : SNS 데이터 변경

공통 권장 함수:

기존 코드 - 로그인 여부만으로 AJAX 호출 가능

```
function btr_popup_save() {
    // current_user_can(), check_ajax_referer() 없음
    $main_popup = get_option('_main_popup', []);
    update_option('_main_popup', $main_popup);
}
add_action('wp_ajax_btr_popup_save', 'btr_popup_save');
```

변경 권장 코드 - 공통 권한/nonce 검증 함수 추가

```
function btr_require_admin_ajax($capability = 'edit_others_posts') {
    check_ajax_referer('btr_admin_nonce', 'nonce');

    if (!current_user_can($capability)) {
        wp_send_json_error(['message' => '권한이 없습니다.'], 403);
    }
}
```

적용 예:

변경 권장 코드 - 각 관리자성 AJAX 시작부에 검증 호출

```
function btr_popup_save() {
    btr_require_admin_ajax('edit_others_posts');

    // 기존 저장 로직
}

function btr_token_get_and_store_token() {
    btr_require_admin_ajax('manage_options');

    // 기존 토큰 발급 로직
}
```

관리자 화면 JS에는 nonce를 전달해야 합니다.

변경 권장 코드 - 프론트/관리자 JS에 nonce 전달

```
wp_localize_script('admin_setting', 'btrAdmin', [
    'ajaxurl' => admin_url('admin-ajax.php'),
    'nonce' => wp_create_nonce('btr_admin_nonce'),
]);
```

5. 악성코드 배포 가능성 및 파일 처리 개선

5.1 공개 업로드 처리

문제: 공개 문의/신고 기능에서 첨부 파일을 웹 접근 가능한 폴더에 저장합니다. 파일 확장자와 실제 MIME이 불일치하거나, 실행 가능한 파일이 저장될 경우 악성 파일 배포 또는 서버 설정에 따른 실행 위험이 발생할 수 있습니다.

권장 수정:

기존 코드 - 사용자 파일명을 이용해 웹 경로에 저장

```
$target_path = ABSPATH . 'wp-content/uploads/contact/' . $file_name;
move_uploaded_file($upload_files['tmp_name'][$key], $target_path);

// base64 업로드도 사용자 제공 fileName 확장자를 사용
$decoded = base64_decode($base64_data);
file_put_contents($target_path, $decoded);
```

변경 권장 코드 - 확장자/MIME/용량 검증 및 안전한 파일명 사용

```

$allowed_mimes = [
    'jpg|jpeg' => 'image/jpeg',
    'png'      => 'image/png',
    'pdf'      => 'application/pdf',
];

$file_check = wp_check_filetype_and_ext(
    $_FILES['upload_file']['tmp_name'],
    $_FILES['upload_file']['name'],
    $allowed_mimes
);

if (empty($file_check['ext']) || empty($file_check['type'])) {
    wp_send_json_error(['message' => '허용되지 않은 파일 형식입니다.'], 400);
}

if ($_FILES['upload_file']['size'] > 10 * 1024 * 1024) {
    wp_send_json_error(['message' => '파일 용량이 초과되었습니다.'], 400);
}

// 파일명은 사용자 입력 대신 서버에서 난수 기반 생성
$safe_name = wp_unique_filename($upload_dir['path'], wp_generate_uuid4() . '.' . $file_check['ext']);

```

추가로 `wp-content/uploads/contact` 경로에는 PHP 실행 차단 설정을 적용합니다.

변경 권장 코드 - 업로드 폴더 내 실행 파일 차단

```

# Apache 예시
<FilesMatch "\.(php|phtml|phar|cgi|pl|asp|aspx|jsp)$">
    Require all denied
</FilesMatch>

```

5.2 download_file 임의 파일 다운로드 가능성

문제: `download_file` 파라미터를 통해 받은 값을 서버 경로로 변환한 후 파일을 내려줍니다. 공격자가 경로를 조작하면 의도하지 않은 서버 파일이 노출될 가능성이 있습니다.

권장 수정 방향:

- 가능하면 `download_file` 핸들러 자체를 제거합니다.
- 다운로드를 게시물 ID 또는 첨부 ID 기반으로만 허용합니다.
- 불가피하게 경로 기반 다운로드가 필요하다면 `realpath()` 결과가 업로드 디렉터리 내부인지 확인합니다.

기존 코드 - 사용자 입력 경로를 readfile로 직접 응답

```

function file_download() {
    $file_url = urldecode($_GET['download_file']);
    $file_path = str_replace(home_url(), ABSPATH, $file_url);

    if (file_exists($file_path)) {
        readfile($file_path);
        exit;
    }
}

```

변경 권장 코드 - 업로드 디렉터리 내부 파일만 허용

```
function secure_file_download() {
    $file_url = isset($_GET['download_file']) ? esc_url_raw(wp_unslash($_GET['download_file'])) : '';
    if (!$file_url) {
        return;
    }

    $upload = wp_upload_dir();
    $base_url = trailingslashit($upload['baseurl']);
    $base_dir = realpath($upload['basedir']);

    if (strpos($file_url, $base_url) !== 0) {
        wp_die('허용되지 않은 다운로드입니다.', 403);
    }

    $relative = ltrim(str_replace($base_url, '', $file_url), '/');
    $file_path = realpath($base_dir . DIRECTORY_SEPARATOR . $relative);

    if (!$file_path || strpos($file_path, $base_dir) !== 0 || !is_file($file_path)) {
        wp_die('파일을 찾을 수 없습니다.', 404);
    }

    // 필요 시 로그인/권한 검증 추가
    readfile($file_path);
    exit;
}
```

6. 공개 조회/삭제 및 비밀번호 처리 개선

`get_safety_single`, `inquiry_search_action`, `user_delete_safety` 는 비로그인 접근이 가능하고 비밀번호 기반 조회/삭제 구조를 사용합니다. 이 방식은 업무 요구상 필요할 수 있으나 무차별 대입 방어와 응답 최소화가 필요합니다.

- 비밀번호는 복호화 가능한 암호화가 아니라 `wp_hash_password()` 또는 `password_hash()` 로 단방향 저장합니다.
- 검증은 `wp_check_password()` 또는 `password_verify()` 로 수행합니다.
- 조회 성공 응답에서도 비밀번호, 내부 상태값, 불필요한 원문 필드는 제외합니다.
- IP, 세션, 문의 ID 기준 실패 횟수 제한을 적용합니다.

기존 코드 - 복호화 가능한 방식으로 비밀번호성 값 저장

```
$safety_password = Crypt::Encrypt($password, $insert_id);
$plain_password = Crypt::Decrypt($saved_password, $id);
```

변경 권장 코드 - 단방향 해시 저장 및 검증

```
$password_hash = wp_hash_password($password);

if (!wp_check_password($input_password, $saved_password_hash)) {
    wp_send_json_error(['message' => '비밀번호가 일치하지 않습니다.'], 403);
}
```

7. XSS 및 출력 인코딩 개선

관리자 화면 JS에서 사용자 입력값을 HTML 문자열로 조합해 출력하는 경우 저장형 XSS 가능성이 생깁니다. 민원 제목, 상세, 이름, 전화번호, 첨부 경로는 모두 신뢰할 수 없는 값으로 보고 출력 시점에 이스케이프해야 합니다.

기존 코드 - 사용자 입력값을 HTML 문자열로 조합

```
const tableRow = `
  <tr>
    <td>${response.title}</td>
    <td>${response.detail}</td>
  </tr>`;
$('.tbody').append(tableRow);
```

변경 권장 코드 - 텍스트 노드로 출력해 스크립트 실행 방지

```
// 권장: 텍스트는 textContent 또는 jQuery .text() 사용
const titleCell = document.createElement('td');
titleCell.textContent = response.title || '';

// URL은 서버에서 esc_url_raw로 저장하고, 출력 시 esc_url 또는 URL 검증 적용
```

8. 조치 우선순위

1. wp_ajax_nopriv_customer_list 제거 및 customer_list 권한 검증 복구.
2. download_file 핸들러 제거 또는 안전한 다운로드 로직으로 교체.
3. 업로드 기능에 서버 측 확장자/MIME/용량 검증과 업로드 폴더 실행 차단 적용.
4. btr_utility, btr_social 관리자성 AJAX 전체에 권한 검증과 nonce 검증 추가.
5. customer_detail, admin_result 의 nopriv 혹 제거.
6. 공개 조회/삭제 기능에 rate limit, 실패 잠금, 비밀번호 해시 저장 적용.
7. phpinfo_temp.php 삭제.
8. 관리자 화면 출력 인코딩 및 JS DOM 출력 방식 개선.

9. 검증 체크리스트

- 비로그인 상태에서 /wp-admin/admin-ajax.php?action=customer_list&division=cyber-audit 호출 시 403 또는 인증 실패가 반환되는지 확인.
- 구독자 등 일반 사용자 계정으로 팝업, 배당정보, 태그, SNS 수정 AJAX 호출 시 403이 반환되는지 확인.
- 관리자 계정과 정상 nonce로 관리자성 AJAX가 기존처럼 동작하는지 확인.
- 허용되지 않은 확장자, MIME 불일치 파일, 용량 초과 파일이 저장되지 않는지 확인.
- download_file 파라미터에 임의 경로를 넣어도 서버 파일이 내려가지 않는지 확인.
- 민원 제목/상세에 HTML 또는 스크립트 문자열을 넣어도 관리자 화면에서 실행되지 않는지 확인.
- 공개 조회/삭제 비밀번호 실패가 일정 횟수 이상 반복될 때 차단되는지 확인.

10. 최종 의견

이번 점검의 핵심은 WordPress 코어 파일 자체의 취약점이 아니라, 커스텀 플러그인이 admin-ajax.php 라우터에 등록된 공개/관리자 AJAX 콜백의 권한 검증 누락입니다. 특히 비로그인 민원 목록 노출은 이미 재현되었으므로 우선 차단해야 합니다. 이후 파일 업로드/다운로드 처리와 일반 사용자 권한 상승 가능 AJAX를 정리하면 주요 위험은 크게 줄어듭니다.